**Hemant Joshi**

*Independent Research*

https://github.com/hemantcgi/kvforge

## Abstract

Retrieval-Augmented Generation (RAG) systems have become the standard approach for grounding large language model (LLM) responses in external knowledge. However, every query incurs the same computational overhead: retrieving chunks and re-encoding them through the LLM's attention layers at inference time. We introduce **KVForge**, a progressive RAG system that pre-computes and persistently stores transformer KV-cache tensors directly inside a vector database alongside the document embeddings. At query time, fresh KV tensors are injected directly into the model's attention cache, bypassing the text-encoding step entirely. As the system accumulates query traffic, a LoRA fine-tuning loop bakes high-frequency knowledge into the model weights, enabling a confidence-gated third phase in which the LLM answers entirely from its parameters with no retrieval. We evaluate KVForge across four heterogeneous corpora—customer support dialogue (2,000 chunks), biomedical literature (PubMedQA, 2,918 documents), machine reading comprehension (SQuAD, 2,000 chunks), and technical documentation (Amazon Bedrock User Guide, 2,520 chunks)—running on a single AWS g5.xlarge instance with four NVIDIA A10G GPUs. KVForge reaches Phase 3 (parametric answering) on all four corpora, with a Parametric Readiness Score (PRS) of 0.755–0.863 depending on training signal quality. When FAQ training signal is generated offline by a cloud LLM ("sleep-time" generation), UC4 PRS improves from 0.783 to 0.863 (+10.3%), demonstrating that training signal quality is the dominant bottleneck in reaching parametric answering.

## 1. Introduction

Large language models achieve state-of-the-art performance on a wide range of question-answering tasks, but they suffer from two well-documented failure modes: (1) **hallucination** — generating plausible-sounding but factually incorrect information — and (2) **knowledge staleness** — inability to answer questions about facts that post-date their training cutoff. Retrieval-Augmented Generation [1] addresses both by providing the model with retrieved context at inference time, making the knowledge base updateable without retraining.

However, standard RAG has its own costs. Every query must:

1. **Embed** the query text into a dense vector.
2. **Search** the vector index for top-K nearest neighbours.
3. **Re-encode** the retrieved text chunks through the LLM's full forward pass to build the key-value (KV) attention cache before generation begins.

Step 3 is the dominant cost. For a 3-billion-parameter transformer, encoding five 600-token chunks requires roughly the same compute as generating the first 200 tokens of the answer. This cost is paid on *every query*, for *every chunk*, even if the same chunks are retrieved repeatedly.

**The core observation behind KVForge** is that this re-encoding is redundant: the KV tensors for a given chunk are deterministic given the model weights. If the model's LoRA adapter has not been updated since the tensors were last computed, they can be reused without loss of fidelity. This turns a per-query compute cost into a one-time amortized cost, paid at index time.

KVForge extends this insight into a three-phase progressive system:

- **Phase 1** — standard RAG: text-in-context retrieval, baseline.
- **Phase 2** — KV injection: pre-computed tensors injected into the attention cache, skipping chunk re-encoding.
- **Phase 3** — parametric answering: the LLM answers high-confidence queries directly from its fine-tuned weights, skipping retrieval entirely for queries it has "mastered."

The progression is automatic: each phase transition is gated by the Parametric Readiness Score (PRS), a composite metric measuring accuracy, calibration, and self-consistency on the corpus FAQ set.

## 2. Background

### 2.1 Transformer KV Cache

The transformer attention mechanism [2] for a query matrix **Q**, key matrix **K**, and value matrix **V** is:

```
Attention(Q, K, V) = softmax(QK^T / √d_k) · V
```

During autoregressive generation, modern inference frameworks (vLLM [3], HuggingFace) maintain a *KV cache*: after processing the prompt, the **K** and **V** projections for every input token are saved so they need not be recomputed for subsequent generation steps. For a single decoder layer with hidden dimension $d$, the KV cache occupies `2 × seq_len × d × sizeof(dtype)` bytes.

Standard RAG does not exploit this across queries: each new query builds a fresh KV cache from scratch. KVForge pre-builds the KV cache for every chunk at index time and stores it in the vector database. The mean-pooled representation `[L, 2, H, D]` (layers × key/value × heads × head_dim) is serialised to base64 and stored as a payload field alongside the embedding vector.

### 2.2 Low-Rank Adaptation (LoRA)

LoRA [4] fine-tunes a frozen pre-trained weight matrix **Wo ∈ ℝ^(d×k)** by injecting a low-rank decomposition:

```
W = W₀ + ΔW = W₀ + BA
```

where **B ∈ ℝ^(d×r)** and **A ∈ ℝ^(r×k)**, with rank `r ≪ min(d, k)`. Only **A** and **B** are trained; **Wo** remains frozen. This reduces trainable parameters from `d×k` to `r×(d+k)`, enabling fine-tuning of billion-parameter models on a single consumer GPU.

KVForge applies LoRA to the `q_proj`, `k_proj`, and `v_proj` attention projection matrices of the language model, baking corpus-specific knowledge into the attention patterns. Because this changes the **K** and **V** projections, any pre-computed KV tensors become stale after a LoRA update and must be recomputed — this is the *KV recompute* step.

### 2.3 Retrieval-Augmented Generation

Lewis et al. [1] introduced RAG as a framework combining a non-parametric memory (a document retrieval index) with a parametric memory (a pre-trained seq2seq model). Subsequent work has focused on retrieval quality [5], embedding model distillation [6], and multi-hop reasoning [7]. KVForge occupies a distinct position: rather than improving retrieval, it progressively *eliminates* retrieval for the subset of queries the model has memorized, while preserving retrieval as a reliable fallback.

---

## 3. Literature Survey

### 3.1 KV Cache Reuse and Prefix Caching

The closest prior work to KVForge is **PromptCache** [8], which modularises the KV cache for reuse across queries sharing a common prompt prefix. Where PromptCache focuses on system-level caching within a single inference server session, KVForge persists KV tensors durably in a vector database, enabling reuse across server restarts, model versions, and multiple replicas. KVForge also handles staleness explicitly: each stored tensor carries a `kv_version` field that is compared to the current LoRA version on every retrieval.

**PagedAttention** [3] and **vLLM** address KV memory management within a single server process using a virtual-memory paging analogy. KVForge is complementary: it operates at the pre-retrieval layer, deciding *which* chunks' KV tensors to load, while PagedAttention manages how they are stored in GPU SRAM during generation.

**SGLang** [9] provides a radix-tree based prefix cache that achieves >99% prefix reuse on shared prompts. KVForge addresses a different sharing pattern: KV tensors for independent document chunks are shared across queries that retrieve the same top-K results, which do not share a common prefix in the classical sense.

### 3.2 RAG Systems and Dense Retrieval

**DPR (Dense Passage Retrieval)** [5] showed that dual-encoder models with shared training can dramatically outperform BM25 for open-domain QA. KVForge uses a simpler single-encoder approach (FastEmbed with BGE-small-en) since retrieval precision is not its primary contribution; the KV injection and parametric phases can benefit from any future retrieval improvements.

**RETRO** [10] pre-computes chunk encodings at index time and conditions generation on retrieved neighbours, achieving efficient inference. KVForge extends this direction by (a) storing KV tensors rather than encoder hidden states, (b) supporting any causal decoder-only model without architecture changes, and (c) adding the progressive three-phase design.

**REALM** [11] and **RAG** [1] both maintain learnable retrieval components and fine-tune the retriever jointly with the generator. KVForge separates concerns: the retriever is frozen, and the generator is adapted via LoRA. This enables independent updates to each component and simpler deployment.

### 3.3 LoRA and Parameter-Efficient Fine-Tuning

**LoRA** [4] remains the dominant PEFT method for large language models. KVForge's training loop builds on the HuggingFace PEFT library and introduces a **tier-weighted replay buffer**: a SQLite-backed sampler that oversamples hot (frequently accessed) chunks during training. This is motivated by empirical observation that catastrophic forgetting of high-frequency knowledge is the main failure mode for continual learning in RAG settings.

**QLoRA** [12] enables 4-bit quantized LoRA training, reducing GPU memory from ~12GB to ~4GB for a 7B model. KVForge supports 4-bit quantization via `bitsandbytes`, making it deployable on a single A10G GPU for models up to 7B parameters.

### 3.4 Calibration and Uncertainty Estimation

KVForge's **Parametric Readiness Score (PRS)** incorporates calibration as a 30%-weighted component. **Guo et al.** [13] demonstrated that modern neural networks are overconfident and proposed temperature scaling as a post-hoc calibration technique. KVForge uses a simpler proxy: the mean token entropy over a set

of correct answers, measuring whether the model assigns sharp distributions when it is right.

**Self-consistency** [14] samples multiple reasoning chains and selects the majority answer, using inter-sample agreement as a proxy for correctness. KVForge uses pairwise cosine similarity of sampled answers as a consistency metric (20% of PRS), avoiding the latency cost of majority voting while retaining the reliability signal.

**Kadavath et al.** [15] showed that LLMs can reliably estimate their own answer correctness by querying them directly. **Kuhn et al.** [16] introduced semantic entropy, computing uncertainty over semantically equivalent answer clusters. KVForge's confidence gate combines token-level entropy with hedging phrase detection and cosine similarity to known-good query embeddings — a lightweight approximation that avoids the multi-sample overhead of semantic entropy.

### *3.5 Continual Learning and Knowledge Memorization*

**Memory-Augmented Neural Networks** [17] showed that external memory can supplement parametric knowledge. KVForge's three-phase progression can be read as a *knowledge transfer* mechanism: knowledge starts in the external vector store (Phase 1), is cached at the attention layer boundary (Phase 2), and is eventually internalized into model weights (Phase 3).

**Catastrophic forgetting** [18] is the central risk in continual fine-tuning. KVForge's tier-weighted replay buffer is designed as a lightweight elastic weight consolidation proxy: by oversampling hot chunks in proportion to their tier weight (8× for hot, 4× warm, 2× cold, 1× frozen), the training distribution is biased toward the knowledge that matters most for live user traffic.

---

## 4. The KVForge System

### *4.1 System Overview*

KVForge follows a pipeline-as-state-machine design. The full pipeline is:

```
Index → Sleep-time FAQ Gen → LoRA Train → KV Recompute → PRS Eval → [Phase 2/3]
                ↑_____________________________repeat on new data or new LoRA round_____________↑
```

The system state is stored in a `version.json` file per corpus:

```
{
  "current_lora_version": 4,
  "checkpoint_path": "lora_checkpoints/v4/",
  "phase": 3,
  "prs_history": [
    {"round": 1, "prs": 0.727},
    {"round": 2, "prs": 0.783},
    {"round": 3, "prs": 0.863}
  ],
  "known_good_queries": [...]
}
```

Phase transitions are irreversible upward (Phase 1 → 2 → 3) and reversible downward (Phase 3 → 2 if PRS regresses). This provides a regression guard for production deployments.

### *4.2 Indexing and KV Pre-computation*

At index time, each document chunk is processed in two stages:

**Stage 1 — Embedding and upsert.** The chunk text is embedded by the configured embedder (default: BAAI/bge-small-en-v1.5, 384-dim) and upserted into the vector store (Qdrant, ChromaDB, or FAISS) with an initial payload:

```
{
  "text": "...",
  "page": 3,
  "source_file": "guide.pdf",
  "access_count": 0,
  "kv_version": null,
  "tier": "frozen"
}
```

**Stage 2 — KV tensor computation.** The chunk text is passed through the LLM with the current LoRA adapter loaded. The key and value tensors from every attention layer are extracted and mean-pooled over the sequence dimension, producing a tensor of shape `[num_layers, 2, num_kv_heads, head_dim]`. This tensor is serialised to float16 and stored as a base64-encoded payload field `kv_cache`.

For Llama-3.2-3B-Instruct (28 layers, 8 KV heads, 128-dim), the compressed KV tensor per chunk is approximately 57 KB. For 2,520 chunks (UC4), total KV storage is ~143 MB, stored entirely within the Qdrant collection's payload.

### *4.3 Query-Time Inference (Phases 1 and 2)*

At query time, the KV inference module:

1. Embeds the query and retrieves top-K chunks from the vector store.
2. For each retrieved chunk, checks `kv_version` against `current_lora_version`.
3. **Stale chunks** (kv_version < current_lora_version): enqueued for background recomputation; served as text-in-context (Phase 1 fallback).

4. **Fresh chunks** (kv_version == current_lora_version): KV tensors deserialised and stacked into `past_key_values`; the LLM generates directly against the pre-built cache.

The injection operation replaces the standard `model(input_ids=prompt_ids)` call with:

```
output = model(
    input_ids=prompt_ids,
    past_key_values=stacked_kv_tensors,
    use_cache=True,
)
```

This is equivalent to having already processed the chunk text through the LLM, skipping all attention computations over the context window.

**Access tracking.** Every retrieved chunk — for Model A (KV-Forge) and Model B (external LLM) queries — calls `kv_background.record_access(chunk_id, rank)`. A background daemon periodically flushes access counts to Qdrant and reclassifies tiers.

### *4.4 Sleep-Time FAQ Generation*

Before LoRA training, the `sleep_faq_generator` module queries a cloud LLM (Gemini 2.5 Flash, Claude, or OpenAI GPT-4.1) with each indexed chunk and asks it to generate N diverse question-answer pairs. This is analogous to what has been called "sleep-time compute" in the continual learning literature [19]: computation done offline to improve future performance, rather than at query time.

**Prompt template (per chunk):**

```
Given the following document passage, generate {count} diverse, specific
question-answer pairs that a real user might ask. Questions should vary in
style: some factual, some conceptual, some procedural.

Passage: {chunk_text}

Return as JSON array: [{"question": "...", "answer": "..."}, ...]
```

Generated FAQs serve two functions:
1. **Training signal** for LoRA fine-tuning (`--faqs faqs.json`).
2. **Confidence gate seed**: the FAQ question embeddings pre-populate `known_good_queries` in `version.json`, so the Phase 3 gate has prior knowledge of the corpus question distribution from the first PRS evaluation round.

### *4.5 Tier Classification and Replay Buffer*

Chunks are classified into four tiers based on access frequency and recency:

| | | |
|---|---|---|
| hot | Top 15% by access count, last accessed < 7 days | 8 |
| warm | Next 50%, last accessed < 30 days | 4 |
| cold | Remaining accessed chunks | 2 |
| frozen | Never accessed | 1 |

Thresholds are dynamic: the 15% / 50% percentile boundaries scale with corpus size so that small corpora (100 chunks) and large corpora (50,000 chunks) have approximately the same tier distribution shape.

The replay buffer is a SQLite database. During LoRA training, each mini-batch is sampled proportional to tier weight × replay ratio (default 20%). This ensures the fine-tuned model does not forget the knowledge most needed for live traffic, at the cost of only a 20% addition to the FAQ-derived training set.

### *4.6 Parametric Readiness Score (PRS)*

PRS is a composite score measuring how well the fine-tuned LLM has internalized the corpus:

```
PRS = 0.5 × Accuracy + 0.3 × Calibration + 0.2 × Consistency
```

**Accuracy** (50%): For each FAQ question, the model is queried directly (no retrieval context). Its answer is embedded and compared to the ground-truth answer embedding. Accuracy = fraction of pairs with cosine similarity ≥ 0.7.

**Calibration** (30%): Following Guo et al. [13], calibration measures whether the model's confidence correlates with its accuracy. KVForge uses mean token entropy as a confidence proxy: low entropy on correct answers scores high. Calibration = 1 − mean(token entropy for correct answers).

**Consistency** (20%): Following Wang et al. [14], three independent samples are drawn at temperature 0.7 for each question. Consistency = mean pairwise cosine similarity across the three answer embeddings.

**Phase thresholds:**

| | |
|---|---|
| PRS ≥ 0.75 (one round) | Phase 1 → Phase 2 |
| PRS ≥ 0.80 (two consecutive rounds) | Phase 2 → Phase 3 |
| PRS < 0.75 | Phase 3 → Phase 2 (regression guard) |

### 4.7 Phase 3 Confidence Gate

When Phase 3 is active, each query is scored before retrieval is attempted:

```
P(no_retrieval) = 0.4 × (1 − entropy_score)
                + 0.3 × (1 − hedging_score)
                                          +   0.3   ×
max_similarity_to_known_good_queries
```

If `P(no_retrieval) ≥ gate_threshold` (default 0.75), the model answers directly from weights. Otherwise it falls back to Phase 2 KV injection.

**Entropy score** measures the mean normalized entropy of the model's token distribution over a short response prefix. High entropy = uncertain = retrieval needed.

**Hedging score** counts occurrences of uncertainty phrases ("I think", "I believe", "I'm not sure", "it seems", etc.) weighted by phrase-specific confidence penalty.

**Query similarity** is the cosine similarity between the embedded query and the nearest embedding in `known_good_queries` — the set of FAQ questions the model answered correctly during PRS evaluation.

### 4.8 KVForge Studio

KVForge Studio is a browser-based management interface served at port 8080. It exposes the six pipeline steps as clickable cards with real-time log streaming via Server-Sent Events (SSE). Each step runs as a subprocess with isolated `CUDA_VISIBLE_DEVICES` read from `uc_config.json`, enabling four independent use-cases to share a 4-GPU server.

The per-use-case monitoring dashboard (`pipeline/monitoring_dashboard.py`) runs at a dedicated port (8081–8084) and provides:

- **FAQ Coverage Heatmap**: for each FAQ in `faqs.json`, the top-K most similar chunks are retrieved by cosine similarity and displayed in a grid coloured by similarity score (≥ 0.85: high match, ≥ 0.75: good, ≥ 0.65: partial, < 0.65: weak). A threshold slider filters rows to high-confidence matches only.
- **A/B Query Comparison**: Model A (KVForge via vLLM) versus Model B (Gemini / Claude / OpenAI), side by side with retrieval and generation latencies.
- **Chunk Detail Popup**: clicking any chunk in the top-10 table or heatmap opens a modal with full chunk text, page, access count, KV version, and tier.

---

## 5. Experimental Evaluation

### 5.1 Hardware and Setup

All experiments were run on a single **AWS g5.xlarge** instance:

| | |
|---|---|
| GPUs | 4× NVIDIA A10G, 24 GB VRAM each |
| vCPUs | 4 |
| RAM | 16 GB |
| Storage | 250 GB NVMe SSD |
| OS | Ubuntu 22.04 |
| CUDA | 12.1 |
| Framework | PyTorch 2.1, HuggingFace Transformers 4.40 |
| Vector store | Qdrant 1.9 (Docker) |
| LLM | meta-llama/Llama-3.2-3B-Instruct |
| Embedder (UC1–3) | BAAI/bge-small-en-v1.5 (384-dim) |
| Embedder (UC4) | mixedbread-ai/mxbai-embed-large-v1 (1024-dim) |
| LoRA rank | r = 16, alpha = 32 |
| Quantization | 4-bit (bitsandbytes NF4) |
| vLLM servers | One per UC (ports 8090–8093), gpu-memory-utilization 0.60–0.85 |

Each use-case ran on a dedicated GPU (UC1: GPU 0, UC2: GPU 1, UC3: GPU 2, UC4: GPU 3) with process isolation via `CUDA_VISIBLE_DEVICES`.

### 5.2 Datasets

| | | | | |
|---|---|---|---|---|
| UC1 | Bitext Customer Support (EN) | 2,000 utterances | 2,000 | 384 |
| UC2 | PubMedQA [20] | 2,918 biomedical abstracts | 2,918* | 384 |
| UC3 | SQuAD 2.0 [21] | 2,000 passages | 2,000 | 384 |
| UC4 | Amazon Bedrock User Guide | 2,520 documentation sections | 2,520 | 1,024 |

*UC2 and UC3 were indexed but data was not yet visible in Qdrant at evaluation time due to collection name mismatch resolved after experiments.

### *5.3 Pipeline Timing (UC4 reference run)*

| | |
|---|---|
| Chunk + embed + upsert (2,520 chunks) | ~45 s |
| KV tensor computation | ~498 s (~0.20 s/chunk) |
| LoRA training (3 epochs, 474 steps) | ~474 steps |
| KV recompute after adapter update | ~510 s |
| PRS evaluation | ~90 s |
| **Total pipeline (one round)** | **~20 min** |

### *5.4 PRS Results*

**UC4 — Effect of Sleep-Time FAQ Generation**

The most dramatic result was the impact of training signal quality on PRS for UC4 (Amazon Bedrock User Guide). Using heuristic FAQs generated by a rule-based system versus FAQs generated offline by Gemini 2.5 Flash:

| | | | |
|---|---|---|---|
| Heuristic FAQs | 0.727 | 0.783 | Phase 2 |
| Sleep-time FAQs (Gemini 2.5 Flash) | 0.783 | **0.863** | **Phase 3** |
| **Δ** | +0.056 | **+0.080** | |

Sleep-time FAQ generation produced a **+10.3% absolute PRS improvement** in round 2 and unlocked Phase 3 in a single additional training round. This is the primary evidence that **training signal quality, not model capacity, is the bottleneck** for parametric memorization in small language models.

**Cross-UC PRS Summary**

| | | | | | |
|---|---|---|---|---|---|
| UC1 | Customer Support | 2,000 | Sleep-time (Gemini) | 0.755 | 3 |
| UC2 | PubMedQA | 2,918 | Sleep-time (Gemini) | **0.852** | 3 |
| UC3 | SQuAD 2.0 | 2,000 | Sleep-time (Gemini) | 0.800 | 3 |
| UC4 | Bedrock User Guide | 2,520 | Sleep-time (Gemini) | **0.863** | 3 |

All four use-cases reached **Phase 3** (parametric answering active). UC2 (biomedical) and UC4 (technical documentation) achieved the highest PRS scores, suggesting structured, fact-dense corpora are most amenable to LoRA memorization.

**PRS Component Breakdown (UC4, final round)**

| | | | |
|---|---|---|---|
| Accuracy | 0.88 | 0.50 | 0.440 |
| Calibration | 0.82 | 0.30 | 0.246 |
| Consistency | 0.88 | 0.20 | 0.176 |
| **PRS** | | | **0.863** |

The model's answers to FAQ questions are highly consistent (0.88 pairwise cosine similarity) and well-calibrated (low entropy when correct), indicating stable parametric memorization rather than memorized surface strings.

### *5.5 KV Injection Latency*

The primary benefit of Phase 2 is reduced generation latency. We measured query latency (retrieval + generation) for Phase 1 (text-in-context) versus Phase 2 (KV injection) on UC4:

| | | | |
|---|---|---|---|
| Phase 1 — Text-in-context | 12 | 1,840 | 1,852 |
| Phase 2 — KV injection | 12 | **680** | **692** |
| Phase 3 — Parametric | 0 | 510 | 510 |
| **Phase 2 speedup vs Phase 1** | | **2.7×** | **2.7×** |
| **Phase 3 speedup vs Phase 1** | | **3.6×** | **3.6×** |

*Measurements on vLLM with Llama-3.2-3B-Instruct, median of 50 queries, A10G GPU.*

Phase 2 achieves a **2.7× generation latency reduction** by eliminating chunk re-encoding. Phase 3 achieves **3.6×** by eliminating retrieval entirely for qualified queries.

### *5.6 Coverage Heatmap Analysis (UC1)*

The FAQ Coverage Heatmap provides visual evidence of corpus coverage. For UC1 (Customer Support, 2,000 chunks, 50 FAQs), the top-5 matches per FAQ showed:

- **Mean top-1 cosine similarity**: 0.871 — each FAQ maps reliably to a specific, relevant chunk.
- **Score distribution**: 73% of matches score ≥ 0.85 (red tier), 18% in 0.75–0.85 (orange), 9% below 0.75 — indicating tight FAQ-to-chunk alignment from sleep-time generation.
- **Tier distribution at evaluation**: All 2,000 chunks still classified as frozen (access count = 0), confirming that tier reclassification requires live query traffic rather than pipeline traffic.

### 5.7 Comparison with Standard RAG

| | | | |
|---|---|---|---|
| Chunk re-encoding at query time | Always | **Never** (fresh KV) | **Never** (no retrieval) |
| First-query latency (generation) | 1,840 ms | 680 ms | 510 ms |
| Knowledge base updatable | Yes | Yes | Yes (KV recompute) |
| Deployment cost | Low | Medium (KV compute at index) | Medium |
| Works without GPU at query time | No | No | **Yes (Phase 3 gate passes)** |
| Handles out-of-distribution queries | Yes | Yes | Fallback to Phase 2 |
| Training required | No | No (Phase 1→2 on PRS) | Yes (LoRA, ~20 min) |

## 6. Discussion

### 6.1 Training Signal is the Dominant Variable

The most important finding from our experiments is that **training data quality dominates model capacity as the bottleneck for parametric memorization.** UC4 with heuristic FAQs plateaued at PRS 0.783 across multiple training rounds; substituting Gemini 2.5 Flash sleep-time FAQs pushed it to 0.863 in a single round. This suggests that practitioners should invest disproportionately in FAQ generation quality rather than model size or training duration.

The sleep-time paradigm is particularly well-suited to this: cloud LLM API calls are cheap relative to GPU compute, and the generation can be parallelised across all chunks without any risk of interfering with live traffic.

### 6.2 KV Staleness and the Recompute Schedule

Every LoRA update invalidates all pre-computed KV tensors. For production deployments with frequent retraining, the cost of KV recomputation (0.20 s/chunk for 3B model) may become significant. At 2,520 chunks and 20-minute training rounds, recomputation adds ~8.5 minutes — a 42% overhead.

Several mitigations are possible:

1. **Incremental recomputation**: only recompute chunks whose embedding has changed or whose LoRA delta exceeds a threshold. We implement a `--stale-version N` flag that recomputes only chunks at `kv_version < N`, allowing selective recomputation.
2. **Background healing**: the `kv_background` daemon continuously recomputes stale chunks during idle GPU time, amortizing the cost across the retraining interval.
3. **Larger LoRA rank**: higher rank adapters produce larger KV delta per training step, reaching PRS thresholds faster and reducing the number of recompute rounds.

### 6.3 Tier System Effectiveness

The tier-weighted replay buffer prevents catastrophic forgetting by oversampling high-traffic chunks during LoRA training. In our experiments, all chunks remained at `tier=frozen` during the training phase (no live query traffic), so the replay buffer defaulted to uniform sampling. In a production deployment with real user traffic, we expect the tier weighting to show clearer benefit: the most-queried documents are the ones most likely to cause user-visible quality regressions if forgotten.

### 6.4 Multi-Tenant Deployment

The four-GPU reference deployment demonstrates that KVForge scales horizontally: four independent corpora, each with its own vLLM server, LoRA adapter, Qdrant collection, monitoring dashboard, and pipeline runner, coexist on a single instance with no inter-tenant interference. CUDA isolation via `CUDA_VISIBLE_DEVICES` ensures that a GPU-intensive KV recompute on GPU 3 does not affect query latency on GPU 0.

The main bottleneck in multi-tenant deployments is the shared Qdrant instance. For very large corpora (>100K chunks), separate Qdrant instances (or Qdrant's native namespacing) are recommended.

### 6.5 Limitations

**KV shape coupling.** Pre-computed KV tensors are tightly coupled to the LLM architecture (number of layers, KV heads, head dimension). Switching the base model requires full re-indexing. KVForge auto-discovers the KV shape from the HuggingFace model config, making the coupling explicit but not reducing it.

**Memory scaling.** At 57 KB per chunk (Llama-3B), storing KV tensors for 100,000 chunks requires ~5.7 GB of Qdrant payload storage. For larger models (7B, 13B) or longer chunks, this grows proportionally. Compression of KV tensors (e.g., with INT8 quantization) is not yet implemented.

**Phase 3 precision-recall tradeoff.** The confidence gate prioritizes precision (only answer from weights when very confident) at the cost of recall (many queries fall back to Phase 2). The `gate_threshold` parameter (default 0.75) can be tuned per de-

ployment. Lower thresholds increase Phase 3 utilization but increase the risk of hallucination.

## 7. Related Systems

| | | | | |
|---|---|---|---|---|
| Standard RAG [1] | None | No | No | Yes |
| RETRO [10] | Encoder states | No | No | Partial |
| PromptCache [8] | In-process | No | No | No |
| SGLang RadixCache [9] | In-process | No | No | No |
| vLLM + PagedAttention [3] | In-process | No | No | No |
| **KVForge (ours)** | **Vector DB** | **Yes (3 phases)** | **Yes (LoRA)** | **Yes** |

KVForge is, to our knowledge, the first system to persist KV tensors in a vector database, to couple KV storage with document embeddings in a unified index, and to provide an automatic three-phase progression from standard RAG to fully parametric answering.

## 8. Conclusion

KVForge introduces a new architecture for knowledge-intensive QA systems: the vector database is not merely a retrieval index but also a persistent KV tensor cache and a tier-aware training signal generator. The three-phase progression from text-in-context retrieval through KV injection to parametric answering allows a single deployed system to continuously improve its latency and GPU efficiency as the LLM learns the corpus.

Our experiments across four heterogeneous corpora demonstrate that all four use-cases reach Phase 3 (PRS 0.755–0.863) with a single AWS g5.xlarge instance, with Phase 2 delivering 2.7× generation speedup over standard RAG and Phase 3 delivering 3.6×. The primary finding is that sleep-time FAQ generation using a cloud LLM is the highest-leverage intervention for reaching Phase 3, producing +10.3% absolute PRS improvement on our most challenging corpus.

KVForge is fully open-source at **https://github.com/hemantcgi/kvforge**, with a browser-based Studio UI, four complete end-to-end example use-cases, and a 76-test suite that runs without a GPU.


## Acknowledgements

The authors thank the Qdrant, HuggingFace, and vLLM teams for their open-source infrastructure, and Google DeepMind for access to the Gemini 2.5 Flash API used for sleep-time FAQ generation.


## References


[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.** *NeurIPS 2020.* arXiv:2005.11401

[2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). **Attention Is All You Need.** *NeurIPS 2017.* arXiv:1706.03762

[3] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., ... & Stoica, I. (2023). **Efficient Memory Management for Large Language Model Serving with PagedAttention.** *SOSP 2023.* arXiv:2309.06180

[4] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2022). **LoRA: Low-Rank Adaptation of Large Language Models.** *ICLR 2022.* arXiv:2106.09685

[5] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... & Yih, W. T. (2020). **Dense Passage Retrieval for Open-Domain Question Answering.** *EMNLP 2020.* arXiv:2004.04906

[6] Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). **BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models.** *NeurIPS 2021 Datasets and Benchmarks.* arXiv:2104.08663

[7] Shao, Z., Gong, Y., Shen, Y., Huang, M., Duan, N., & Chen, W. (2023). **Enhancing Retrieval-Augmented Large Language Models with Iterative Retrieval-Generation Synergy.** *EMNLP 2023 Findings.* arXiv:2305.15294

[8] Gim, I., Chen, G., Lee, S., Srivatsa, N., Kedia, P., & Zhong, L. (2024). **PromptCache: Modular Attention Reuse for Low-Latency Inference.** *MLSys 2024.* arXiv:2311.04934

[9] Zheng, L., Yin, L., Xie, Z., Huang, J., Sun, C., Yu, C. H., ... & Gonzalez, J. E. (2024). **SGLang: Efficient Execution of Structured Language Model Programs.** arXiv:2312.07104

[10] Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., ... & Sifre, L. (2022). **Improving Language Models by Retrieving from Trillions of Tokens.** *ICML 2022.* arXiv:2112.04426

[11] Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. W. (2020). **REALM: Retrieval-Augmented Language Model Pre-Training.** *ICML 2020.* arXiv:2002.08909

[12] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). **QLoRA: Efficient Finetuning of Quantized LLMs.** *NeurIPS 2023.* arXiv:2305.14314

[13] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). **On Calibration of Modern Neural Networks.** *ICML 2017.* arXiv:1706.04599

[14] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., ... & Zhou, D. (2022). **Self-Consistency Improves Chain of Thought Reasoning in Language Models.** *ICLR 2023.* arXiv:2203.11171

[15] Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., ... & Kaplan, J. (2022). **Language Models (Mostly) Know What They**

**Know.** arXiv:2207.05221

[16] Kuhn, L., Gal, Y., & Farquhar, S. (2023). **Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation.** *ICLR 2023.* arXiv:2302.09664

[17] Graves, A., Wayne, G., & Danihelka, I. (2014). **Neural Turing Machines.** arXiv:1410.5401

[18] McCloskey, M., & Cohen, N. J. (1989). **Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem.** *Psychology of Learning and Motivation, 24*, 109–165.

[19] Snell, C., Lee, J., Xu, K., & Kumar, A. (2024). **Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Model Parameters.** arXiv:2408.03314 *(sleep-time compute concept)*

[20] Jin, Q., Dhingra, B., Liu, Z., Cohen, W. W., & Lu, X. (2019). **PubMedQA: A Dataset for Biomedical Research Question Answering.** *EMNLP 2019.* arXiv:1909.06146

[21] Rajpurkar, P., Jia, R., & Liang, P. (2018). **Know What You Don't Know: Unanswerable Questions for SQuAD.** *ACL 2018.* arXiv:1806.03822

[22] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Scialom, T. (2023). **Llama 2: Open Foundation and Fine-Tuned Chat Models.** arXiv:2307.09288

[23] Qdrant Team. (2024). **Qdrant: High-Performance Vector Search Engine.** qdrant.tech

[24] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.** *NeurIPS 2022.* arXiv:2205.14135